Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
SAN PEDRO CAMPUS
PUP, Main Road, Barangay United Bayanihan, City of San Pedro, Laguna

UniSync Database Roles and Privileges

Activity 4 Progress Documentation

1. Introduction

This document presents the progress and implementation of Activity 4: Database Users, Roles, Privileges, and Access Control for the UniSync project.

The activity focuses on creating database roles, assigning appropriate privileges, and testing access restrictions using PostgreSQL. The goal is to apply the principle of least privilege, where each database user receives only the permissions needed for their assigned responsibilities.

2. Project Overview

The project used for this activity is UniSync, which has a PostgreSQL database named grp10_unisyncdb.

The database contains the following tables:

- account
- organization
- membership
- event
- venue
- event_schedule
- registration
- notification

The database was initialized using the project's SQL scripts and populated with sample records for testing.

3. Database Inspection

Before creating the roles, the database structure and existing records were inspected.

The database contains:

- 8 tables
- 8 sequences
- 1 schema named public

The following row counts were recorded:

Table	Number of Records
account	130
organization	30
membership	239
event	50
venue	50
event_schedule	119
registration	945
notification	0

The inspection helped identify the existing database objects before assigning permissions.

4. Role Creation and Configuration

Three group roles and three login roles were created.

4.1 Group Roles

Group Role	Purpose
unisync_admin_grp	For database administration tasks
unisync_runtime_grp	For normal application operations

unisync_readonly_grp	For users who only need to view approved data

All group roles were created with the NOLOGIN attribute. This means they cannot be used directly to connect to the database.

4.2 Login Roles

Login Role	Assigned Group Role
unisync_dba	unisync_admin_grp
unisync_app	unisync_runtime_grp
unisync_auditor	unisync_readonly_grp

- The login roles were created with the LOGIN attribute and assigned to their corresponding group roles.
- Passwords were configured locally and were not included in the repository.

5. Privilege Assignment

Privileges were assigned according to each role's responsibility.

5.1 Database and Schema Privileges

All three group roles were granted CONNECT permission on `grp10_unisyncdb` and USAGE permission on the `public` schema.

To restrict database access, the CONNECT privilege was revoked from PUBLIC on `grp10_unisyncdb`. Explicit CONNECT privileges were retained for the three intended group roles.

The CREATE privilege was also revoked from PUBLIC on the `public` schema. Only the administrative group was granted CREATE permission, while the runtime and read-only groups received USAGE permission only.

5.2 Administrative Role

The unisync_admin_grp role was granted all privileges on the existing tables and sequences in the public schema.

This includes permissions such as SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, and TRIGGER on tables.

5.3 Runtime Role

The unisync_runtime_grp role was granted:

- SELECT
- INSERT
- UPDATE

It was also granted USAGE and SELECT privileges on the sequences.

The runtime role was intentionally not granted DELETE permission to prevent the application account from directly deleting database records.

5.4 Read-Only Role

The unisync_readonly_grp role was granted SELECT permission on seven tables:

- organization
- membership
- event
- venue
- event_schedule
- registration
- notification

For the account table, access was limited to these five columns:

- account_id
- account_name
- email
- avatar_url
- account_status

The password_hash column was excluded to protect sensitive account information.

5.5 Default Privileges

- Default privileges were also configured for future tables and sequences created by the postgres role in the public schema.
- The administrative group receives all privileges, while the runtime group receives SELECT, INSERT, and UPDATE on tables and USAGE and SELECT on sequences.
- No automatic SELECT privileges were assigned to the read-only group for future tables.

6. Access Control Testing

The three login roles were tested separately to verify that their assigned privileges worked as expected.

6.1 Runtime Account — unisync_app

The following tests were performed:

Operation	Result
SELECT	Successful
INSERT	Successful
UPDATE	Successful
DELETE	Denied

SELECT successfully retrieved the account records. INSERT and UPDATE operations were tested inside transactions and rolled back afterward to avoid permanently changing the sample data.

The DELETE test returned a permission-denied error, confirming that the runtime role cannot directly delete records.

6.2 Read-Only Account — unisync_auditor

The following tests were performed:

Operation	Result
SELECT approved account columns	Successful
SELECT password_hash	Denied
INSERT	Denied
UPDATE	Denied
DELETE	Denied

The auditor account successfully accessed the approved account columns but could not access the password_hash column.

Its SELECT privileges on the seven other approved tables were also verified through PostgreSQL privilege information.

6.3 Administrative Account — unisync_dba

- The administrative account successfully connected to the database and retrieved the account record count.
- INSERT and UPDATE operations were tested inside transactions and rolled back afterward.
- A DELETE statement targeting a nonexistent account ID returned DELETE 0. This did not remove any records and did not produce a permission error.
- The account also successfully created and dropped its own test table, public.activity4_admin_test, in the public schema. This verified its schema CREATE privilege, but did not demonstrate ownership-level operations on existing tables.

6.4 Testing Summary

The tests confirmed the following:

- The administrative account has the assigned table privileges and schema CREATE permission.
- The runtime account can SELECT, INSERT, and UPDATE records but cannot DELETE them.
- The auditor account can only read approved data and cannot access the protected password_hash column.
- The three accounts have the intended group role memberships.

6.5 PHP Application Integration Testing

The UniSync PHP application was successfully connected to the PostgreSQL database using the unisync_app runtime account.

The following application workflows were tested successfully:

- **Account Registration:** Successfully created a user account.
- **User Login:** Successfully logged in using the registered account.
- **Event Booking:** Successfully completed an event booking through the application.

These tests confirmed that the runtime account could perform the database operations required by the tested application workflows.

Limitation: The local XAMPP environment uses PHP 8.0.30, while the project specifies PHP 8.1 or newer. Despite this version difference, the tested application workflows worked successfully in the local environment.

7. Issues and Limitations

During the implementation of the database roles and privileges, several setup considerations and limitations were encountered.

7.1 PostgreSQL Environment Setup

PostgreSQL 18.6 was installed and configured locally. The UniSync database was successfully created, and its initialization and sample data scripts were executed. The database tables, sequences, and sample records were also verified.

7.2 Role Configuration and Permissions

The group roles and login accounts were successfully created and configured. Their memberships and privileges were checked using PostgreSQL verification queries.

One important consideration was that granting table privileges to the administrative role does not automatically transfer ownership of existing tables. The existing tables remained owned by postgres, so ownership-dependent operations were not included in the admin account tests.

7.3 PHP Application Integration

The UniSync PHP application was successfully connected to the PostgreSQL database using the unisync_app runtime account.

The local XAMPP environment initially had PHP 8.0.30 without the enabled PostgreSQL PDO extension. The pdo_pgsql extension was enabled through the PHP configuration file, allowing the application to connect to PostgreSQL.

The following application workflows were successfully tested:

- **Account Registration:** Successfully created a user account.
- **User Login:** Successfully logged in using the registered account.
- **Event Booking:** Successfully completed an event booking through the application.

These tests confirmed that the application could perform the required database operations using the restricted runtime account.

Limitation: The local environment still uses PHP 8.0.30, while the project specifies PHP 8.1 or newer. The tested workflows worked, but the PHP version requirement remains a compatibility consideration.

7.4 Default Privileges

Default privileges were configured specifically for objects created by the postgres role in the public schema. These settings apply to future objects created by that role and do not automatically update the privileges of existing objects.

The read-only group was not given automatic SELECT privileges on future tables to avoid exposing potentially sensitive information.

Despite these limitations, the database role configuration and direct access control tests were completed successfully.

8. Evidence and Repository

The SQL script and supporting screenshots for the UniSync access-control implementation are stored in the group's Car Rental Management System repository under db-access-control/Group10/.

Repository links:

- **Repository:** https://github.com/123airabels/Car_Rental_Management_System
- **UniSync access-control folder:** https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10
- **SQL script:** https://github.com/123airabels/Car_Rental_Management_System/blob/main/db-access-control/Group10/unisync-roles-and-privileges.sql
- **Evidence folder:** https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence

8.1 Evidence Screenshots

The evidence folder contains screenshots organized into the following categories:

1. [01]role_config.png
2. [02]Role_Memberships/
3. [03]Database_and_schema privileges/
4. [04]Table_and_column privileges/
5. [05]Role_security_settings/
6. [06]Runtime_account_tests/
7. [07]Auditor_account_tests/
8. [08]Admin account tests/
9. [09]Sequence_privileges/
10. [10]create_database_restriction.png
11. [11]unrelated_database_access_denied.png
12. [12]car_rental_runtime_connection.png
13. [13]create_role_restriction.png
14. [14]runtime_delete_restriction.png
15. [15]auditor_modification_restrictions.png
16. [16]final_role_security_verification.png
17. [17]php_database_connection.png

Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
SAN PEDRO CAMPUS
PUP, Main Road, Barangay United Bayanihan, City of San Pedro, Laguna

8.2 GitHub Upload

The evidence screenshots were committed and pushed to the main branch.

Commit: 7089f08
Commit Message: Update UniSync access control and add security test evidence

The SQL script and additional security-test screenshots were committed and pushed successfully to the repository.

9. SQL Scripts Summary

The unisync-roles-and-privileges.sql script contains the SQL commands used to configure database roles, assign privileges, and apply access restrictions for the UniSync database.

The script covers the following operations:

- **Group role creation** – Created unisync_admin_grp, unisync_runtime_grp, and unisync_readonly_grp as NOLOGIN roles.
- **Login role creation** – Created unisync_dba, unisync_app, and unisync_auditor as LOGIN roles.
- **Role membership** – Assigned each login role to its corresponding group role.
- **Database access** – Revoked CONNECT from PUBLIC on grp10_unisyncdb and granted CONNECT to the three intended group roles.
- **Schema privileges** – Revoked CREATE from PUBLIC on the public schema, granted USAGE to all three group roles, and granted CREATE only to the admin group.
- **Admin privileges** – Granted all privileges on existing tables and sequences in the public schema to unisync_admin_grp.
- **Runtime privileges** – Granted SELECT, INSERT, and UPDATE on existing tables, plus USAGE and SELECT on sequences, to unisync_runtime_grp.
- **Auditor privileges** – Granted SELECT on approved tables and selected account columns, excluding sensitive account information such as password_hash.
- **Default privileges** – Configured privileges for future tables and sequences created by postgres in the public schema, so the admin and runtime groups receive their intended access.
- **Access-control verification** – Used privilege-checking queries and actual login tests to verify permitted and denied operations.

The script follows the principle of least privilege by limiting each role to the permissions required for its assigned responsibility. Passwords are configured separately and are not stored in the SQL script.

10. Verification Results

The database roles and privileges were checked using PostgreSQL queries and direct login tests.

10.1 Runtime Account Verification

Account	Operation	Expected Result	Actual Result
unisync_app	SELECT account records	Allowed	Successful; retrieved 130 records
unisync_app	INSERT	Allowed	Successful; transaction rolled back
unisync_app	UPDATE	Allowed	Successful; transaction rolled back
unisync_app	DELETE	Denied	Permission denied
unisync_app	CREATE DATABASE	Denied	Permission denied

Evidence:
https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence/[06]Runtime_account_tests

https://github.com/123airabels/Car_Rental_Management_System/blob/main/db-access-control/Group10/evidence/[10]create_database_restriction.png

10.2 Read-Only Account Verification

Account	Operation	Expected Result	Actual Result
unisync_auditor	SELECT approved account columns	Allowed	Successful
unisync_auditor	SELECT password_hash	Denied	Permission denied
unisync_auditor	INSERT	Denied	Permission denied

unisync_auditor	UPDATE	Denied	Permission denied
unisync_auditor	DELETE	Denied	Permission denied

The SELECT privileges on the seven other approved tables were also checked through PostgreSQL privilege information.

Evidence:
https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence/[07]Auditor_account_tests

10.3 Administrative Account Verification

Account	Operation	Expected Result	Actual Result
unisync_dba	Connect to UniSync database	Allowed	Successful
unisync_dba	SELECT account count	Allowed	Successful; retrieved 130 records
unisync_dba	INSERT	Allowed	Successful; transaction rolled back
unisync_dba	UPDATE	Allowed	Successful; transaction rolled back
unisync_dba	DELETE nonexistent account ID	No matching record	Returned DELETE 0
unisync_dba	Create and drop own test table	Allowed	Successful

The DELETE test did not independently verify DELETE permission because the target account ID did not exist. The administrative table privileges were separately verified through PostgreSQL privilege information.

Evidence:
https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence/[08]Admin%20account%20tests

10.4 Additional Privilege Verification

The following configurations were also checked:

- Group roles have the NOLOGIN attribute.
- Login roles have the LOGIN attribute.
- Login roles belong to their intended group roles.
- Administrative, runtime, and read-only roles have their assigned database and schema privileges.
- Runtime and read-only roles do not have schema CREATE permission.
- The auditor has no sequence usage privilege.
- The auditor's account table access is limited to the five approved columns.

Evidence:

- https://github.com/123airabels/Car_Rental_Management_System/blob/main/db-access-control/Group10/evidence/[01]role_config.png
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence/[02]Role_Memberships
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence/[03]Database_and_schema%20privileges
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence/[04]Table_and_column%20privileges
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence/[05]Role_security_settings
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group10/evidence/[09]Sequence_privileges

10.5 Additional Security Verification

The following additional tests were conducted to verify database isolation and prevent unauthorized administrative operations:

Test Account	Operation	Expected Result	Actual Result
unisync_app	CREATE DATABASE activity4_test_db;	Denied	Denied
unisync_app	CREATE ROLE activity4_test_role;	Denied	Denied
unisync_app	Delete a record from account	Denied	Denied

unisync_auditor	Update a record in organization	Denied	Denied
unisync_auditor	Delete a record from organization	Denied	Denied
unisync_app	Connect to car_rental_db	Denied	Denied
app_runtime	Connect to car_rental_db	Allowed	Allowed

The PUBLIC CONNECT privilege was revoked from both grp10_unisyncdb and car_rental_db. Explicit database access for the intended group roles was retained and verified.

The company_db database was also observed to allow connections through its existing PUBLIC CONNECT privilege. Its permissions were left unchanged because it may be used by another project.

These results demonstrate that the tested UniSync runtime account cannot create databases or roles, perform unauthorized modifications, or connect to the tested unrelated Car Rental database.

11. Issues and Fixes

The following issues and limitations were encountered during the UniSync database setup and access-control implementation.

Issue	Action Taken / Resolution	Status
PHP initially lacked the PostgreSQL PDO extension, preventing the application from connecting to the database.	Enabled the pdo_pgsql extension in the XAMPP PHP configuration and updated the Apache-served .env file to use the unisync_app runtime account. Successfully tested account registration, user login, and event booking.	Resolved; PHP version limitation remains
Existing database tables were owned by postgres.	Privileges were granted to the appropriate group roles. Existing table ownership was retained.	Addressed through grants
The auditor needed limited access to account information.	Column-level SELECT privileges were granted only to approved columns in the account table.	Resolved
The runtime account needed to perform normal database operations without administrative permissions.	SELECT, INSERT, and UPDATE privileges were assigned. DELETE and schema CREATE permissions were not granted.	Resolved
Future database objects needed appropriate access permissions.	Default privileges were configured for tables and sequences created by postgres in the public schema.	Configured
The administrative DELETE test returned DELETE 0.	The test used a nonexistent account ID, so it did not independently confirm the DELETE permission. The assigned privileges were checked separately through PostgreSQL privilege information.	Additional testing recommended

11.1 Additional Verification Completed

The following additional checks were completed after the initial access-control tests:

1. Verified that unisync_app cannot create a new database.
2. Verified that unisync_app cannot create a new PostgreSQL role.
3. Verified that unisync_app cannot connect to the unrelated car_rental_db database after PUBLIC CONNECT was revoked.
4. Confirmed that the intended Car Rental runtime account, app_runtime, can still connect to car_rental_db.
5. Reviewed the relevant role memberships and database privileges to check that the intended accounts retained their access.

The company_db database was also checked and found to allow connections through PUBLIC CONNECT. Its permissions were left unchanged because it may be used by another project.

These checks provide additional evidence of access restrictions and database isolation. However, access to every unrelated database on the PostgreSQL server has not been verified.

Additional Issue: Unrestricted Database Connection Through PUBLIC

Issue: During cross-database access testing, the unisync_app account was initially able to connect to car_rental_db because the database still allowed connections through the PUBLIC role.

Cause: PostgreSQL grants CONNECT to PUBLIC by default unless that privilege is revoked or otherwise restricted.

Fix: The following command was executed while connected to a different database:

REVOKE CONNECT ON DATABASE car_rental_db FROM PUBLIC;

Explicit CONNECT privileges for the intended Car Rental group roles were retained. Subsequent tests confirmed that unisync_app could no longer connect, while app_runtime could still connect successfully.

Additional observation: The company_db database also allowed connections through PUBLIC. Its permissions were not changed because it may belong to another project. Therefore, database isolation was verified for the tested Car Rental database, not every database on the PostgreSQL server.

12. Feedback to the Owning Group

The following observations and recommendations were identified while setting up and testing the UniSync database.

Area	Observation	Recommendation
Database setup	The database and sample records were successfully inspected and prepared for access-control testing.	Provide clear setup instructions and indicate the required PostgreSQL version.
Database privileges	Access restrictions were implemented using separate admin, runtime, and readonly roles.	Maintain role-based permissions and avoid granting unnecessary privileges to application accounts.
Sensitive information	The auditor role was restricted from viewing the password_hash column.	Continue limiting access to sensitive account information.
Application integration	The PHP application was successfully connected to PostgreSQL using the unisync_app runtime account. Account registration, login, and event booking were tested successfully.	Upgrade the local PHP environment from 8.0.30 to the project's required PHP 8.1 or newer version to maintain compatibility.
Documentation	The database setup and access-control steps were documented during testing.	Include clear instructions for creating roles, configuring permissions, and testing database access.

13. AI Usage Declaration

AI tools, including ChatGPT, were used as assistance during the UniSync database setup and access-control implementation.

The AI assistance was used for the following tasks:

- Understanding PostgreSQL roles, memberships, privileges, and access-control concepts.
- Reviewing and improving SQL scripts for role creation and privilege assignment.
- Explaining PostgreSQL permission errors and helping troubleshoot database access issues.
- Preparing verification queries and identifying appropriate allowed and denied access tests.
- Organizing the progress documentation and explaining the security test results.

All SQL commands and configurations were reviewed and tested using PostgreSQL 18.6 through psql. Actual database outputs and screenshots were used as evidence. Suggestions that were not tested were identified as pending rather than reported as completed.

AI was used as a support tool. The group reviewed the implementation and verified the recorded results against the actual database environment.

14. Conclusion

The UniSync database was successfully prepared for Activity 4 access-control testing using PostgreSQL 18.6. Three group roles and their corresponding login roles were created to separate administrative, application runtime, and read-only responsibilities.

Privileges were assigned based on the principle of least privilege. The admin role received the necessary administrative permissions, the runtime role was limited to its required data operations, and the auditor role was restricted to approved tables and account columns. Sensitive information, including `password_hash`, was excluded from the auditor's access.

Verification included checking role memberships, database and schema privileges, table and column permissions, sequence privileges, and actual allowed and denied operations. Additional tests confirmed that the runtime account could not create databases or roles, and that database connection restrictions worked for the tested accounts.

The UniSync PHP application was also successfully connected to PostgreSQL using the `unisync_app` runtime account. Account registration, user login, and event booking were tested successfully, confirming that the application could perform the tested database operations.

One limitation is that the local XAMPP environment uses PHP 8.0.30, while the project specifies PHP 8.1 or newer. The tested workflows worked in the current environment, but the PHP version requirement remains to be addressed for compatibility.

The SQL script and supporting screenshots were saved in the group's repository for documentation and reference. Overall, the activity provided practical experience in PostgreSQL role management, privilege assignment, access restriction, and verification.